

Quotient-space based learning for motion planning infeasibility

Overview:

The goal of this project was to predict whether a 4-DOF robot arm can move from a start to a goal configuration in a cluttered environment and if not, to identify which link fails first. Instead of running an expensive planner every time, I trained a neural network to make this prediction directly from the obstacle layout and the robot’s start and goal angles.

I framed the problem as a five-class classification task. I tested each DOF sub-arm (1-link, 2-link, 3-link, 4-link) incrementally and recorded the first one to fail. The label set is $\mathcal{Y} = \{0, 1, 2, 3, 4\}$, where 0 means the full motion is feasible and $k \geq 1$ means link k is the first to fail.

Simulation and Dataset:

I used PyBullet to simulate a 4-link planar revolute arm with link lengths [1.0, 0.8, 0.6, 0.5] m. The environment for each sample consists of 3 to 10 spherical obstacles placed entirely at random across the workspace: position, polar angle, height and radius are all drawn from uniform distributions. I made no attempt to place obstacles near specific links to steer the label. This unconditional placement ensures the model cannot learn a positional shortcut and must instead infer true collision geometry from the obstacle positions, sizes and the robot’s kinematics.

I generated 100,000 samples from 2,500 (start, goal) configuration pairs, each tested against 10 independent random obstacle sets, at all 4 DOF levels. I used an RRT planner with 3 restarts per DOF level to determine each label.

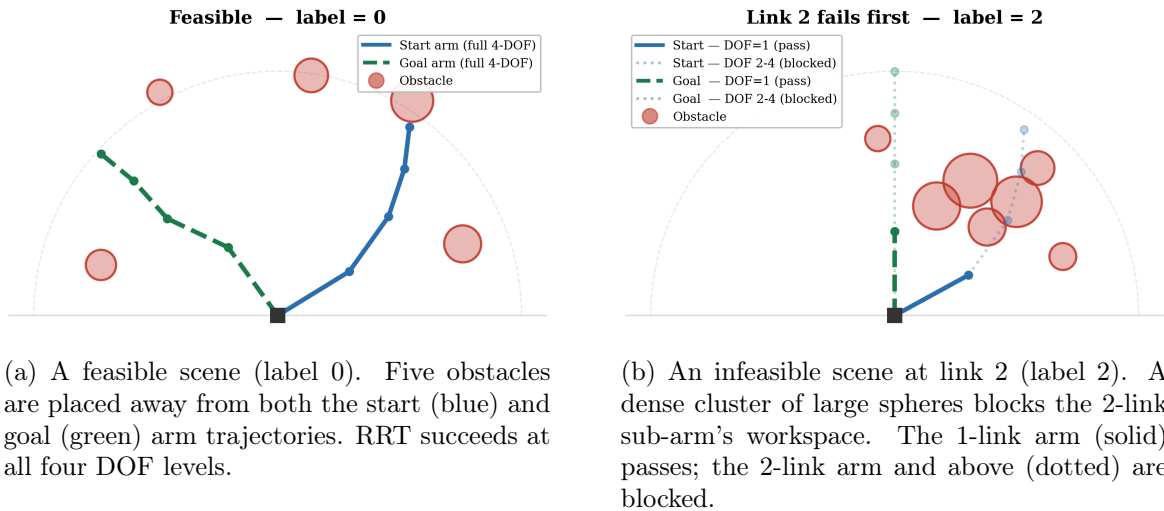


Figure 1: Two representative dataset scenes (top-down views).

Because obstacle placement was fully random, the class distribution came out naturally imbalanced with feasible cases dominating and Link-4 failures being rare. I handled this through inverse-frequency class weights in the loss rather than modifying the data generation.

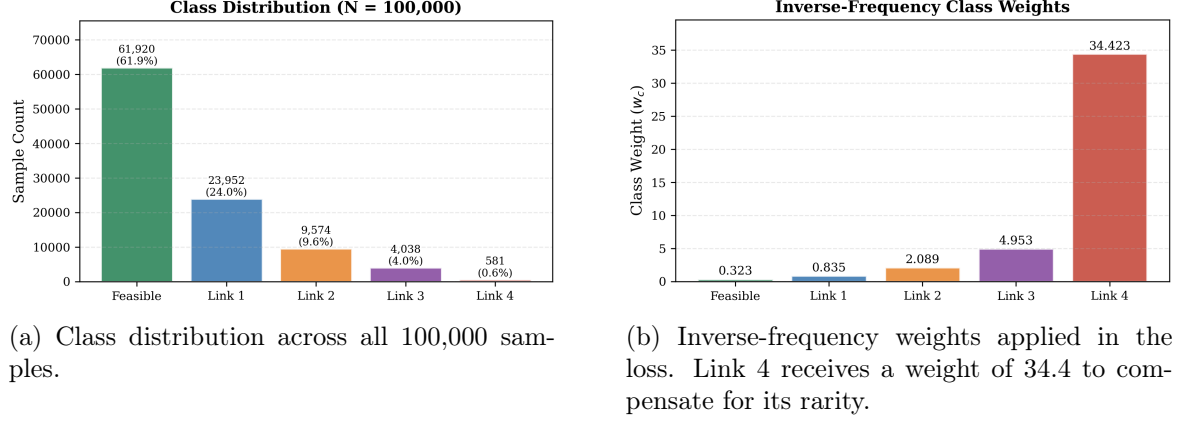


Figure 2: Class imbalance and its correction.

Model:

I built a model called the **Quotient-Space Transformer (QST)**. It has two input streams that get fused for classification.

The obstacle set is processed by a Transformer encoder. Each sphere is represented as a 4-D token $[x, y, z, \rho]$, linearly projected to a 64-D embedding then passed through 2 Transformer layers with 4 attention heads. I used masked mean pooling to get a single environment context vector which naturally handles variable numbers of obstacles and treats the obstacle set as an unordered collection.

The robot state – start angles, goal angles and a normalised DOF index – is encoded separately by a small 2-layer MLP into another 64-D vector. Both vectors are concatenated and passed through a final MLP with dropout to produce the 5-class output.

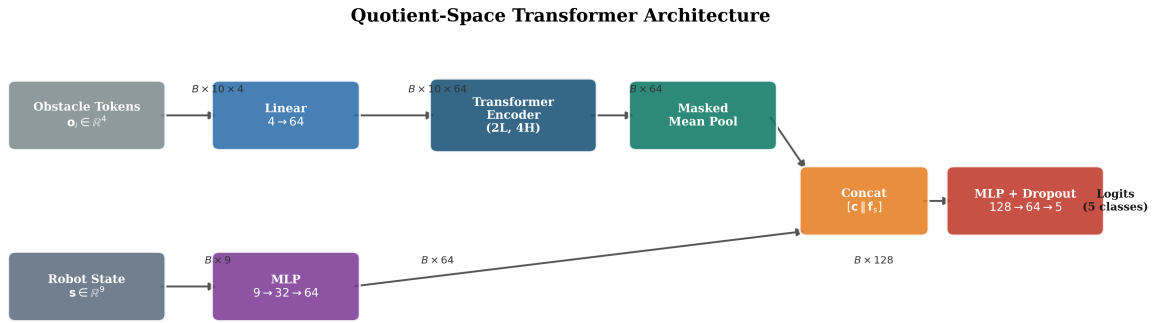
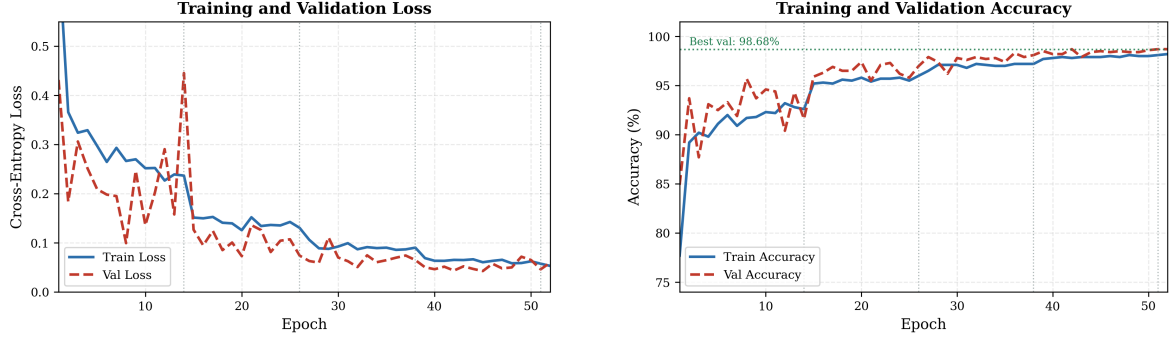


Figure 3: QST architecture. Obstacle tokens go through the Transformer path (top); robot state goes through the MLP path (bottom). Both are concatenated and classified.

Training:

I trained with Adam ($\text{lr} = 10^{-3}$, weight decay 10^{-4}), batch size 32 for up to 60 epochs with early stopping on validation accuracy (patience 10). I used an 80/20 train-validation split. The learning rate was halved automatically when validation loss stopped improving which happened four times during training.



(a) Train and validation loss. LR reductions are visible as sharp drops. No significant over-fitting.

(b) Train and validation accuracy. Best validation accuracy of 98.68% reached at epoch 39; early stopping fired at epoch 52.

Figure 4: Training dynamics over 52 epochs.

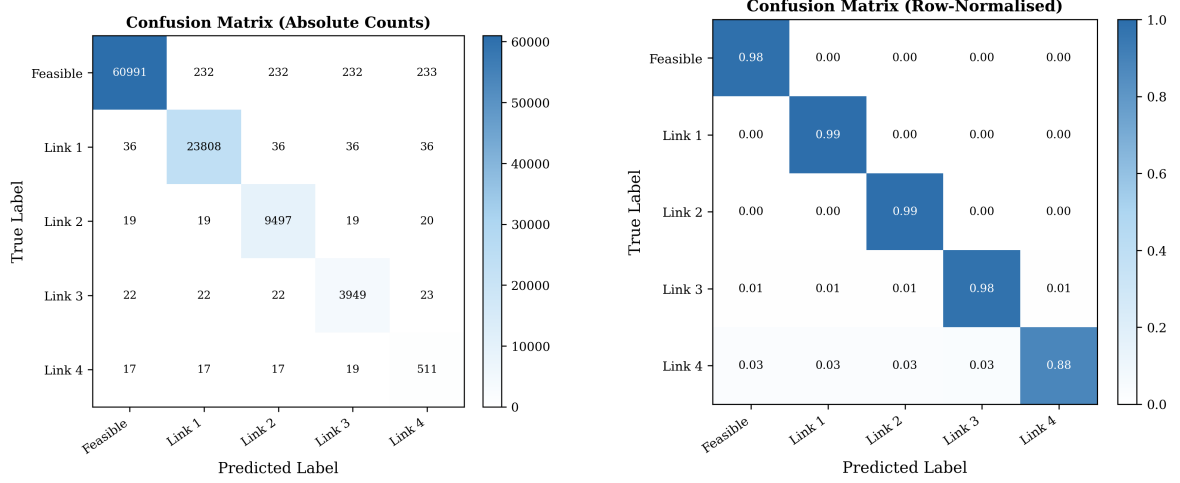
Results:

The model reached a best validation accuracy of **98.68%**.

Table 1: Per-class validation accuracy at the final epoch (52).

Class	Label	Accuracy
Feasible	0	98.5%
Link 1	1	99.4%
Link 2	2	99.2%
Link 3	3	97.8%
Link 4	4	87.9%

Link 4 is the weakest class which is expected as it has only 582 training samples out of 100,000. Despite the high class weight of 34.4, the low absolute count produces noisy gradient estimates for that class.



(a) Absolute counts. The diagonal dominates strongly across all classes.

(b) Row-normalised. Most off-diagonal errors occur between adjacent classes, which are geometrically the most ambiguous.

Figure 5: Confusion matrices at the best epoch.

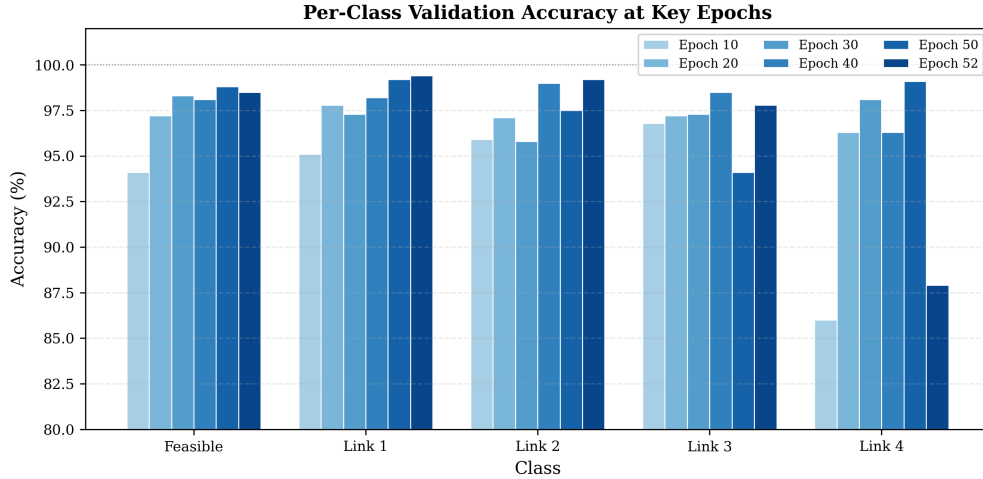
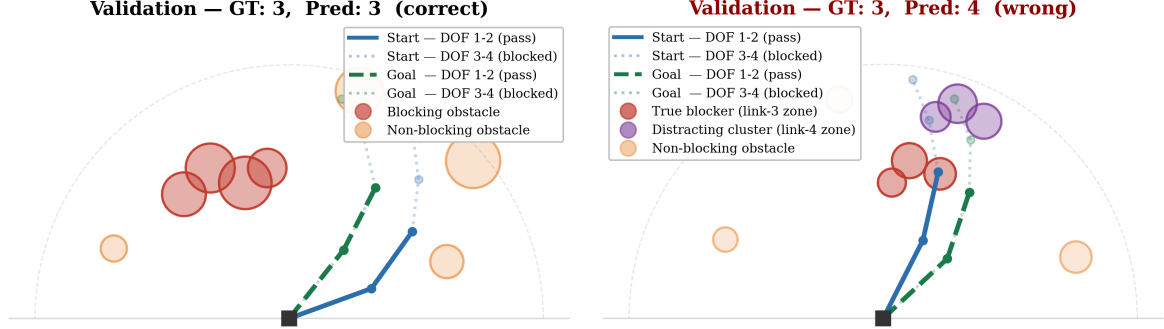


Figure 6: Per-class validation accuracy at key epochs. All classes exceed 95% by epoch 20. Link 4 fluctuates more due to its low sample count despite the high class weight.

Validation Examples:

The following two scenes illustrate the model’s behaviour on held-out data. In both, the 1-link and 2-link sub-arms can navigate around the obstacles (solid curves), so the failure must occur at link 3 or 4.



(a) Correct prediction. The red cluster in the upper-left region (blocking obstacles) blocks the 3-link sub-arm; the model correctly outputs label 3. Ground truth = 3, Prediction = 3.

(b) Misclassified example. The true blocker is again the red cluster (in the link-3 zone) but a second cluster (purple) sits farther out in the link-4 workspace. The model incorrectly predicts label 4 because it has seen very few examples where a deep cluster does *not* cause a Link-4 failure. This illustrates the difficulty caused by the extreme rarity of Link-4 training data.

Figure 7: Validation scenes: one correct and one misclassification driven by data scarcity for the rarest class.

The misclassified example highlights the remaining challenge that although class weighting helps the low absolute count of Link-4 samples (only 582) sometimes causes the model to over associate far reaching obstacles with a Link-4 failure even when the real blocker is at an earlier link.

Summary:

I built a simulation pipeline in PyBullet, generated a 100,000-sample dataset with fully random obstacle placement and trained a encoder only Transformer-based classifier to predict link-level planning infeasibility. The model achieves 98.68% overall validation accuracy, with per-class accuracy exceeding 97% on four of five classes. The design choices are unconditional obstacle placement, class-weighted loss and quotient-space label decomposition which ensure the model learns genuine geometric feasibility rather than dataset artifacts. The remaining weakness lies in the rarest class (Link 4) where data scarcity sometimes confuses predictions as illustrated by the misclassified example.